

The Dev Tool: a glass-box agent over Father Solanus Casey’s archive

Solanus Project Pipeline — step_7

Contents

1	TL;DR	1
2	The mental model: one settings object, made visible	1
3	A brief jargon primer	2
4	The swappable axes (the variables)	3
5	The toggleable tools (the agent’s powers)	3
6	The agent loop and the router	4
7	Citations: the IIIF modal + PDF download	5
8	The knowledge-graph view	6
9	Observability and evaluation	6
9.1	Per-request observability (the “under the hood” panel)	6
9.2	Offline evaluation: which knobs actually win?	6
10	How to run it	7
11	Caveats	8
12	Sources / where to read next	8

1 TL;DR

The one-sentence version

The dev tool is a web app that answers questions about Father Solanus Casey’s ~570 letters and ~716 notebook pages with **grounded, cited** prose — and lets you **watch every step the machine took**, while swapping the model that did it. It is a glass box, not a black box.

- **Three model choices are *variables***, picked from dropdowns: the **LLM** (generation), the **embedding space** (retrieval), and the **reranker**. Swap one, re-ask, compare. That loop is the whole instrument.

- **Retrieval is a row of toggles** — BM25, dense, rerank, HyDE, multi-query, the LLM router. In the agent path the same idea becomes **toggleable tools**: `vector_search`, `pdf_fulltext_search`, `entity_lookup`, `graph_query`, `temporal_query`, `community_summary`.
- **Every claim cites the exact handwritten region**. Click a [1] and a deep-zoom IIIF viewer opens the scan, boxes the cited region, and offers a *Download source PDF* button.
- **A knowledge-graph view** renders people, places, records and their relations; clicking a record opens its scan.
- **Observability + evaluation are first-class**: every request returns a trace and a running cost ledger; an offline harness scores whole *configurations* on RAGAS-style metrics so you pick a winner empirically.
- **Nothing bills you by accident**. The local path costs \$0; every paid touchpoint routes through `lib/costlog.py` and is gated.

2 The mental model: one settings object, made visible

The cleanest way to hold the whole tool in your head is this: **the left rail of the UI is a live picture of one `RetrievalConfig`** (the same dataclass `lib/retrieval.py` uses). Every dropdown and every toggle does exactly one thing — it sets a field on that object. When you press **Ask**, the object is serialized and POSTed to the backend; the configured retrieval runs *once*; an LLM writes a cited answer; and the page renders four things at once: the answer, its citations, the trace, and the cost.

Why this design earns its keep

Because the only thing that changes between two questions is the *settings object*, you can do real science: hold the question fixed, flip one toggle or swap one model, ask again, and read the difference straight off the trace and the eval. The tool is built so that “change one variable, observe” is the *easiest* thing to do.

There are two answer paths, and the server (`app/server.py`) speaks both, because they answer two different research questions.

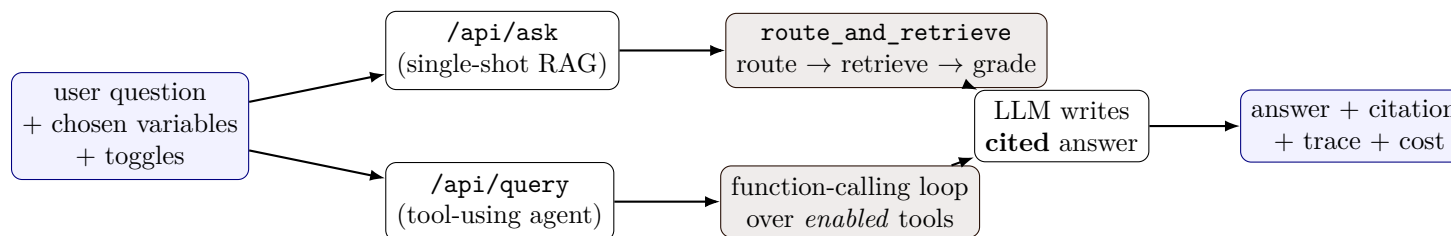


Figure 1: Two paths, one discipline. `/api/ask` runs a single toggle-driven retrieval then synthesizes; `/api/query` lets the model loop over toggleable tools. Both ground every claim, abstain when evidence is thin, cite `doc_id+rid+page+vertices`, and cost-log every model touch.

3 A brief jargon primer

Skip if RAG, an embedding space, a reranker, and function-calling are already old friends.

Retrieval-Augmented Generation (RAG), in one breath

A language model is fluent but forgetful and prone to confident invention. RAG fixes that by *retrieving* the relevant passages from a trusted corpus first, then asking the model to answer *only from those passages*. For an archive this is non-negotiable: we want the friar’s own words, not the model’s paraphrase of the internet.

Embedding space. An *embedding* turns a passage into a vector (a point in high-dimensional space) so that similar meanings sit near each other. A *space* is a particular (model, dimension) pair, e.g. `gemini-embedding-001@1536`. Dense retrieval = “find the passages whose vectors point in nearly the same direction as the question’s vector” (cosine similarity). Different spaces draw the map differently, which is exactly why we make the space a swappable variable and *measure* which one is best for *this* corpus.

BM25 (lexical). The classic keyword score: a passage ranks high if it literally contains the query’s rare words. It is unbeatable for exact strings — a name like *O’Donnell*, a year like *1933* — and it is free and local. We fuse it with dense retrieval (via reciprocal-rank fusion) so we get both “means the same thing” and “says the same word”.

Reranker. A fusion list is good but coarse. A *cross-encoder* reranker reads the question and each candidate *together* and re-orders the top handful for a sharper final ranking. It is the most accurate and the most expensive step per item, so it is off by default and a toggle.

HyDE / multi-query. Two cheap recall boosters. **HyDE** embeds a *hypothetical answer* (the model imagines what a good answer looks like, and we search with *that*), which often lands closer to the real passage than the terse question does. **Multi-query** (RAG-Fusion) asks the same thing a few different ways and fuses the results — handy when notebook entries are written in one register and your question in another.

Function-calling (tool use). Modern LLMs can be handed a menu of “tools” (each a name + description + JSON parameter schema). Instead of answering immediately, the model can *call* a tool; we run it, hand back the result, and let the model continue. The agent path is just this menu plus a loop.

Self-RAG / CRAG grade, and abstention. Before trusting retrieved context, we *grade* it: does it actually support an answer? The verdict is **answer**, **caveat**, or **abstain**. In an archive, “*I could not find this*” is a correct and valuable answer, so abstaining on weak evidence is a feature, not a failure.

4 The swappable axes (the variables)

The UI does not hard-code its options; it builds itself from `GET /api/config`, which reports the available LLMs (`config.LLMS`), **only the embedding spaces that have a built partition**

(`vectorstore.spaces()`), and the rerankers (`config.RERANKERS`). That last point is a small honesty win: the embedding dropdown can never offer a partition you cannot actually search.

The human picks the variables; the model never does

In the agent loop, `_inject_request_variables` threads the chosen embedding space / reranker into `vector_search`'s arguments and the chosen LLM into `community_summary` — *behind* the model's back. So the model's entire decision surface is “what should I search for?”, never “which embedding model should I use?”. Shrinking the model's choices is what makes its tool-use reliable.

5 The toggleable tools (the agent's powers)

Each capability lives in its own file under `app/tools/` and registers a single `Tool` — a name, the description *the model reads to decide when to call it*, a JSON-Schema for its arguments, and the Python callable. The payoff of one-file-per-tool: **adding a power is dropping a file that calls `register(...)`**, and it shows up everywhere (UI switch, function-calling schema, trace) with zero edits to the server.

The registered tools (`app/tools/`)

tool	answers	default	cost
<code>vector_search</code>	“which passages match?”	on	free local; hosted = paid
<code>pdf_fulltext_search</code>	exact-string page search (FTS5)	on	free, local
<code>entity_lookup</code>	name → canonical entity	on	free, local
<code>temporal_query</code>	dates / timelines (EDTF)	on	free, local
<code>graph_query</code>	relational hops on the KG	off	free local (needs graph)
<code>community_summary</code>	whole-corpus themes	off	free digests; summary = paid

The two `default_on=False` tools are off *honestly*: their artifacts (`data/graph.json`; the community index) must be built first, and “an honest UI never offers a switch that can't fire.” A tool that raises never kills the turn — `tools.call(...)` wraps every invocation and returns a structured `{ok:false, error:...}` the agent and the trace panel can read and recover from.

A concrete Solanus example

Ask “Which favors were reported for cancer?” with `vector_search` on. The agent searches *cancer cure reported*, and gets back notebook entries each carrying provenance — e.g. `doc_id=Volume_6__p178`, `rid=doc_1.src_content.0`, `pdf_page=183`, and the gold polygon vertices — then answers with `[1]...[n]` markers. Now toggle on `graph_query` and ask “What continues onto page 178?": the agent walks a `CONTINUES_ON` edge and cites *both* fragments. Same archive, a different power switched on.

6 The agent loop and the router

The function-calling loop in `agent_query` is small on purpose:

1. Seed the conversation with the question; the **system prompt** carries the grounding + citation + abstain policy.
2. Ask the model, offering **only the enabled tools**’ declarations. It either *calls* tools or *answers*.
3. For each call: run the tool (error-wrapped), **record a trace span**, **harvest provenance into citations**, remember the best retrieval **grade**, append the result to history.
4. Repeat until it answers or hits **max_steps** — a safety cap so a confused model cannot loop forever and run up cost.

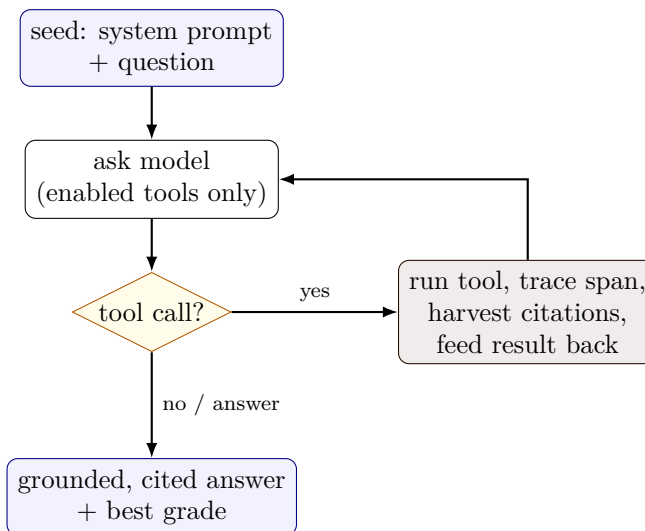


Figure 2: The agent loop. The cycle back to “ask model” is capped by **max_steps** so cost is always bounded.

In the single-shot path the equivalent “thinking” is the **adaptive router**: `route_and_retrieve` classifies the query’s complexity and picks a strategy (simple → direct lexical; complex → fan-out + fuse), then the CRAG-style grade decides *answer* / *caveat* / *abstain*. The trace shows the route decision on its own line, so you can see *why* the system spent (or chose not to).

7 Citations: the IIIF modal + PDF download

This is the heart of “provenance-first.” A citation here is not a footnote string; it is a **clickable pointer to pixels**. Clicking [1] (or a source row, or a record node in the graph) calls `GET /api/region?doc_id=...&rid=...`, which the server resolves from the `step_6` source records: the region’s **text**, its gold polygon **vertices**, the OCR **min_conf**, the page **PNG** (`/api/image/...`), the **searchable PDF** (`/api/pdf/...`, preferring the text-layer copy), and the **IIIF manifest** (`/api/manifest/...`). The front-end then deep-zooms the scan with OpenSeadragon, converts the pixel **vertices** into viewport coordinates, *draws a highlight box* (dimming the rest of the page), frames the region, and wires the *Download source PDF* button.

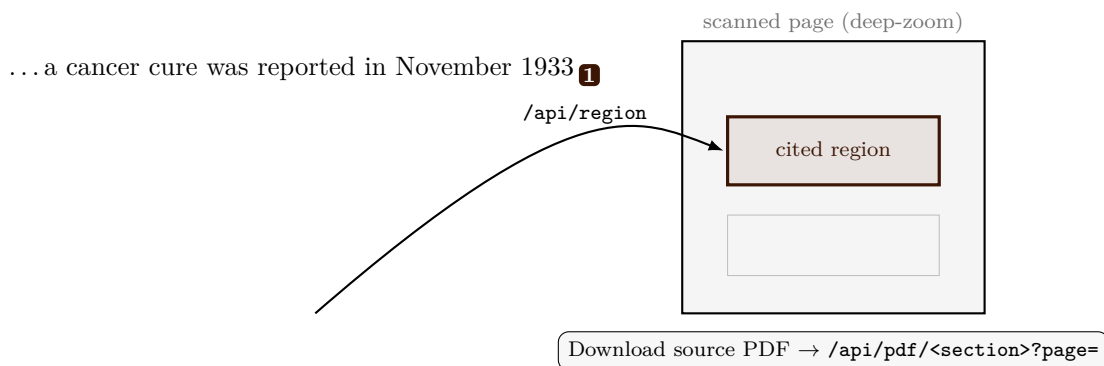


Figure 3: Click [1] → `/api/region` returns text + polygon + image/PDF/manifest URLs → OpenSeadragon boxes the exact handwritten region and the footer links the page’s searchable PDF.

Two safety habits worth copying

The modal **degrades gracefully**: if the backend is offline or a PDF was never generated, it shows what the citation already carries plus a friendly note, instead of breaking. And it **never innerHTMLs model output** — the LLM’s text is untrusted, so [n] markers are parsed into DOM chips to avoid an XSS hole. An archive tool that mishandles untrusted text is a liability, not a convenience.

8 The knowledge-graph view

GET `/api/graph` serves `data/graph.json` (built on demand if networkx is present). The UI renders it with Cytoscape.js: nodes are letters, notebook pages/entries, and resolved entities (person / place / org / condition / outcome / event), each brand-tinted by kind; edges are asserted relations (`WROTE_TO`, `LOCATED_AT`, `CONTINUES_ON`, ...). There is an *entities-only* filter, *fit / reload* controls, and a legend. Clicking an **entity** prints an inspect line (label, aliases, mention count); clicking a **record** node opens its scan in the *same* citation modal. The graph is therefore a navigable index back into the page images — not a decorative blob.

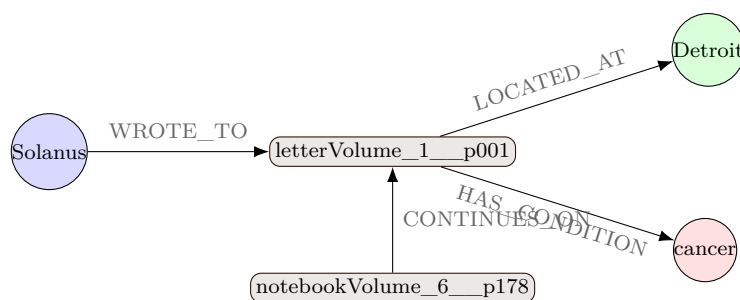


Figure 4: A few hops of the archival graph. Click “cancer” to inspect the entity; click “notebook Volume_6__p178” to open its scan — the same modal the citations use.

9 Observability and evaluation

9.1 Per-request observability (the “under the hood” panel)

The tool’s whole pitch is “see under the hood,” so every request returns a **trace**. Ideally those are OpenTelemetry spans flowing to **Arize Phoenix** (OpenInference is the LLM-semantics layer Phoenix reads); if that stack imports, the server registers a tracer provider and an exporter configured via the standard `OTEL_*` env vars picks the spans up. But we refuse to *depend* on a heavy optional stack — so if it is absent, a dependency-free `_JsonTracer` records every step (`agent_start`, each `llm_turn`, each `tool_call` with latency, `agent_end`), returns it *inline* as the response **trace**, and appends one JSONL line to `app/traces/agent_traces.jsonl`. Either way the panel works with zero extra deps, and tracing is best-effort so it can never break a request.

9.2 Offline evaluation: which knobs actually win?

Once you can retrieve and answer, the hard question is *which configuration to ship*. `eval/eval.py` is the scoreboard. It runs a set of **golden questions** through the real stack and scores the four metrics the RAG-eval literature has settled on.

The four metrics, intuition first (the RAGAS/ARES playbook)

- **Faithfulness** — “does the answer actually follow from the retrieved context, or did the model invent it?” Decompose the answer into atomic claims; score = supported claims / all claims. In an archive an unsupported claim is the cardinal sin, so this is the headline number.
- **Context precision** — of the passages we retrieved, what fraction are relevant? (Don’t drown the generator in noise.)
- **Context recall** — of the passages we *should* have retrieved, what fraction did we get? (Don’t miss the evidence.)
- **Answer relevancy** — does the answer address the question that was *asked* (vs. true-but-off-topic)?

A configuration *passes* only if its mean clears *every* threshold — intentionally strict, because we would rather flag a borderline config than ship a confabulating one.

Thresholds (RESEARCH_PLAN Part E)

metric	pass bar
faithfulness	≥ 0.75 (headline)
context precision	≥ 0.70
context recall	≥ 0.80
answer relevancy	≥ 0.70

The harness scores these for **every** {LLM $\times$ embedding $\times$ reranker} combination (`-sweep`) and writes a ranked leaderboard to `data/eval_results.json` with a per-config verdict. There are **two grading backends**: a deterministic *free heuristic* (the default — claim-splitting + lexical-overlap support, \$0) and a *RAGAS-faithful LLM judge* (paid, double-gated behind `-use-llm-judge`

-i-accept-cost). An ARES-style *synthetic* question generator exists too, but — per project rule that gold is David’s authority — it writes to a separate review file and is *never* auto-merged into the gold set.

10 How to run it

```
# from the repo root -- add the two web deps to the step_7 venv (one-time)
pipeline_v3/step_7/venv/bin/pip install "fastapi>=0.110" "uvicorn[standard]>=0.29"

# (optional) observability -- else the JSON-tracer fallback is used automatically
# pip install opentelemetry-sdk opentelemetry-api openinference-instrumentation arize-
  phoenix

# launch the API + UI (this SERVES the app; do not run it as part of a build)
pipeline_v3/step_7/venv/bin/uvicorn app.server:app --reload --port 8000
# -> open http://localhost:8000

# FREE offline self-check (no server, no paid call) -- confirms what is wired:
pipeline_v3/step_7/venv/bin/python app/server.py

# FREE eval (heuristic metrics, default config); --sweep compares all combos:
pipeline_v3/step_7/venv/bin/python eval/eval.py --sweep
```

11 Caveats

- **fastapi + uvicorn are required and not yet installed.** The server imports them lazily; if missing, `app.server:app` becomes a stub that returns an actionable 500 telling you what to install — so `import app.server` still works for tests/inspection.
- **/api/query and /api/ask make paid LLM calls** (the agent reasons; the synthesizer writes). Retrieval is free with the local embedding + reranker variables; a hosted embedding space, a paid reranker, or any paid toggle (rerank / HyDE / multi-query / LLM-router) spends — and is cost-logged. Nothing bills until you ask.
- Some tools **depend on upstream artifacts**: `graph_query` needs `data/graph.json` (run `stages/build_graph.py`, install `networkx`); `temporal_query` needs `data/dates.json`; `community_summary`’s paid path needs the community index. These default OFF or degrade to a well-formed empty result.
- OpenTelemetry/Phoenix is **optional**; without it you still get the full inline JSON trace, just not the exportable Phoenix timeline.
- The eval metrics are **proxies on the free path** (lexical overlap, `expected_where` slices); treat the free leaderboard as a relative A/B and enable the gated LLM judge for sharper absolute numbers.

12 Sources / where to read next

- **Backend**: `app/server.py` (agent loop, `/api/*`, region resolution, observability fallback); `app/tools/base.py` (the Tool contract + registry); the six tool modules in `app/tools/`.

- **Front-end:** `app/static/index.html` (the five panes); `app/static/app.js` (rail → request, citation modal, graph); `app/static/theme.css`.
- **Retrieval brain + eval:** `lib/retrieval.py` (`RetrievalConfig`, `route_and_retrieve`, the grade); `eval/eval.py` (RAGAS/ARES metrics + the sweep); `eval/golden_questions.json`.
- **Foundation:** `config.py` (the model registry / variables); `lib/costlog.py` (the ledger every call routes through); `lib/providers/llm.py` + `embed.py` (model-agnostic adapters); `lib/chunks.py` + `lib/vectorstore.py` (data shapes + spaces).
- **Context:** `RESEARCH_PLAN.md` (Part D retrieval frontier, Part E agent/eval/observability); `STYLE_GUIDE.md`.